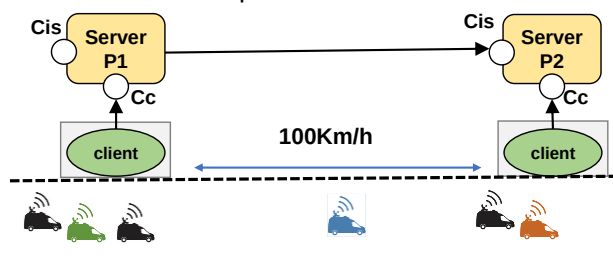


EXAME ÉPOCA NORMAL (2ª PARTE, COM CONSULTA, DURAÇÃO: 75 min., 9 valores)

Nota: Nas perguntas seguintes, quando solicitado, deve usar troços de código Java essencial, podendo intermediar com linhas de comentário. Escrever código irrelevante com definições de *import* de *packages*, *static main*, *System.out.println*; *public* etc. é perda de tempo que lhe pode faltar para aspetos mais importantes.

9. [3 val.] Considere um cenário de controlo de excesso de velocidade média entre os pontos P1 e P2 de uma estrada.

Cada veículo tem um dispositivo eletrónico que ao passar em determinados pontos (P1 e P2 na figura) envia por uma rede sem fios a sua matrícula para um equipamento (fora do âmbito da questão). Esse equipamento executa um cliente que comunica com um servidor gRPC que disponibiliza o contrato Cc. O cliente envia para o servidor uma mensagem com matrícula do veículo e o valor de relógio (hh:mm:ss) de passagem do veículo. Cada servidor, por exemplo o existente no ponto P1, reenvia a mensagem (matrícula, hora) para o servidor seguinte (P2 na figura) que disponibiliza um contrato entre servidores (Cis).



Quando um veículo passa no segundo servidor é registada da mesma forma a matrícula e a hora de passagem que permitirá calcular se o veículo ultrapassou a velocidade média (na figura 100 Km/h). Embora fora do âmbito desta questão, se o veículo ultrapassar essa velocidade média o servidor regista numa base de dados esse facto para que posteriormente o proprietário do veículo seja notificado pela contraordenação.

Considerando o *middleware* gRPC, proponha os contratos (Cc e Cis) em *protobuf*, os pressupostos das configurações, bem como o esboço em Java das aplicações (*Server* e *Client*), nomeadamente as classes necessárias. Em particular apresente a funcionalidade da operação do servidor que recebe a mensagem do cliente e envia a notificação para o servidor seguinte.

10. [3 val.] Considere uma empresa financeira de comercialização de ativos financeiros, nomeadamente *crypto* moedas, ações etc. Nessa empresa existem dezenas de funcionários a analisar e a dar respostas aos milhares de pedidos diários dos clientes. Esses pedidos podem indicar uma das seguintes 4 operações: Compra; Venda; Troca; Criação de NFT (*non-fungible tokens*). Utilizando unicamente o *middleware* RabbitMQ, proponha a arquitetura (*Exchanges*, *Queues*, respetivos *Bindings* e *Routing Keys* necessários) de um sistema com os seguintes requisitos:

- Associado a cada operação podem existir múltiplos funcionários normais, em número variável ao longo dos dias. Por exemplo, nalguns dias é normal existir mais pedidos sobre compras, exigindo mais funcionários;
- Quando um funcionário considera que um pedido é complexo, por exemplo criação de NFT ou Troca de ativos, reenvia o pedido para um departamento especial, onde outros funcionários especializados tratam desse pedido;
- Para efeitos de registo, controlo de qualidade e possíveis futuras auditorias, todas as operações pedidas pelos clientes são guardadas num ficheiro de *Logging*;

Apresente os troços de código Java essenciais para a aplicação do cliente que submete pedidos e as aplicações usadas pelos funcionários (*NormalApp* e *EspecializadoApp*) que analisam e tratam as várias operações pedidas pelos clientes da empresa financeira.

11. [3 val.] [Não precisa apresentar código] Considere que usa o *middleware* de comunicação por grupos (*Spread Toolkit*) conjuntamente com o *middleware* gRPC no desenvolvimento de uma aplicação Cliente/Servidor, onde se pretende ter um grupo de múltiplos servidores em *load balancing* que permite distribuir a carga dos múltiplos pedidos pelos múltiplos servidores com os seguintes requisitos:

- Todos os clientes e todos os servidores pertencem a um grupo de nome Q11.
- Após se juntar ao grupo, o cliente envia um pedido prévio ao grupo Q11 para lhe ser devolvido o par (IP, port) do servidor gRPC, ao qual deverá enviar o pedido com os dados que pretende processar;
- De seguida o cliente envia o pedido para o servidor gRPC

Tendo em conta que deve existir uma distribuição de pedidos (*load balancing*) pelos vários servidores, descreva uma estratégia baseada num diagrama de interação, nas estruturas de dados e nas mensagens necessárias, para que os servidores decidam entre si por consenso qual o servidor que responde ao pedido prévio do cliente com seu par (IP, port), para que lhe possa ser submetido, via gRPC, o pedido de processamento de dados.